

JAVA INTERVIEW PREPARATION

CHAPTER 20

SETS AND ORDERED COLLECTIONS

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Set Implementations and Ordered Collections: Uniqueness, Ordering, and Equality Semantics

Senior / Lead Java Developer Reading Chapter

A Set expresses uniqueness, but each implementation defines how uniqueness is detected and what ordering, performance, memory, and concurrency behavior accompanies it. Choosing HashSet because it is familiar can silently discard order. Choosing TreeSet can silently discard objects when its comparator says they are equal. Senior developers make those semantics part of the domain design.

HashSet Is Backed by HashMap

HashSet commonly delegates storage to a HashMap. Set elements become map keys and a shared placeholder object becomes the value.

```
HashSet<CustomerId>
  |
  v
HashMap<CustomerId, PRESENT>
```

Therefore HashSet inherits the essential hashing rules: equal elements must have equal hash codes, elements should not mutate equality-relevant state while stored, expected contains/add/remove is $O(1)$, collision behavior matters, and iteration order is unspecified.

```
Set<String> permissions = new HashSet<>();
boolean first = permissions.add("ORDER_READ"); // true
boolean second = permissions.add("ORDER_READ"); // false
```

add returns whether the set changed. That is often cleaner than contains followed by add.

Uniqueness Is a Business Decision

A Set is only as correct as its element equality. If Customer equality includes every mutable profile field, two objects for the same customer may be treated as different after an email change. If equality uses only a stable customer ID, the set models identity more reliably.

Before choosing a Set, state the uniqueness key:

```
email addresses: normalized address?
customers: customer ID?
money: amount plus currency?
requests: client ID plus idempotency key?
```

Normalization must occur consistently. `User@example.com` and `user@example.com` are equal only if the domain defines and implements that rule.

LinkedHashSet: Predictable Encounter Order

LinkedHashSet adds a linked ordering structure to hash-table membership. It normally preserves insertion order while retaining expected constant-time membership.

```
Set<String> stages = new LinkedHashSet<>();
stages.add("validate");
stages.add("price");
stages.add("persist");
// iteration: validate, price, persist
```

The order costs additional links per entry. Use it when deterministic output, stable tests, or first-seen deduplication has value. Do not pay for ordering that no consumer observes.

```
List<Event> distinctInArrivalOrder =
    new ArrayList<>(new LinkedHashSet<>(events));
```

This deduplicates according to Event equality, not according to an arbitrary field. For deduplication by ID while retaining the first or last event, an explicitly merged LinkedHashMap is often clearer.

TreeSet: Sorted Uniqueness

TreeSet is normally backed by a TreeMap implemented as a balanced red-black tree. contains, add, and remove are $O(\log n)$, and iteration follows natural or comparator order.

```
Set<Job> jobs = new TreeSet<>(
    Comparator.comparing(Job::priority)
                .thenComparing(Job::createdAt)
                .thenComparing(Job::id)
);
```

TreeSet uses comparison result zero to determine set uniqueness. It does not separately call equals to decide whether both elements may remain.

```
Comparator<Customer> byLastName =
    Comparator.comparing(Customer::lastName);

// Two different customers named Smith compare as zero.
// A TreeSet using this comparator retains only one of them.
```

Add stable tie-breakers until comparison zero represents the intended uniqueness. Ideally ordering is consistent with equals, though specialized sets may intentionally document different semantics.

Comparator Correctness

A comparator must be antisymmetric, transitive, and stable for elements while they are stored. Arithmetic subtraction is unsafe because overflow can reverse ordering.

```
// Wrong: overflow is possible
Comparator<Task> bad = (a, b) -> a.priority() - b.priority();

// Correct
Comparator<Task> good = Comparator.comparingInt(Task::priority);
```

Mutating an ordering field after insertion can make an element appear in the wrong tree location, just as mutating a hash field damages HashSet lookup. Sorted-set elements should have stable comparison state.

NavigableSet Operations

TreeSet implements NavigableSet, providing floor, ceiling, lower, higher, and range views.

```

NavigableSet<Instant> schedule = new TreeSet<>();
Instant next = schedule.ceiling(now);
Instant previous = schedule.lower(now);
SortedSet<Instant> window = schedule.subSet(start, true, end, false);

```

Range operations are powerful for time windows and ordered indexes. Their returned sets are usually backed views. Changes affect the original set, and additions outside the permitted range fail.

EnumSet: A Compact Domain Set

EnumSet is specialized for enum values and is commonly represented as a bit vector. It is compact, fast, type-safe, and iterates in enum declaration order.

```

EnumSet<Permission> permissions = EnumSet.of(
    Permission.READ,
    Permission.WRITE
);

if (permissions.contains(Permission.WRITE)) {
    updateOrder();
}

```

Prefer EnumSet over HashSet<Enum> for flags and fixed domain categories. It rejects null and requires the enum type to be known, including for an empty set through `EnumSet.noneOf(Permission.class)`.

Immutable and Unmodifiable Sets

Set.of creates an unmodifiable set and rejects duplicate elements and nulls.

```

Set<String> roles = Set.of("ADMIN", "AUDITOR");

```

`Collections.unmodifiableSet(existing)` creates a read-only view; changes to the backing set remain visible. `Set.copyOf(existing)` creates an unmodifiable set from the current elements, though contained mutable objects remain mutable.

```

unmodifiable view -> same backing collection can still change
immutable snapshot -> membership cannot change through original collection
deep immutability -> element state is also immutable

```

Make the intended guarantee explicit in APIs.

Concurrent Set Choices

`ConcurrentHashMap.newKeySet` creates a scalable concurrent set with `ConcurrentHashMap` semantics and no nulls.

```

Set<String> activeRequestIds = ConcurrentHashMap.newKeySet();
activeRequestIds.add(requestId);

```

`ConcurrentSkipListSet` provides sorted concurrent access with expected logarithmic operations and weakly consistent iteration. `CopyOnWriteArraySet` fits very small, read-mostly sets such as listeners, where each write can copy the entire backing array.

A concurrent set makes membership operations safe; it does not make a check across several sets or a mutable stored element atomic.

Set Algebra

Set operations express many business rules clearly:

```
Set<Permission> effective = EnumSet.copyOf(userPermissions);
effective.retainAll(rolePermissions); // intersection

Set<String> missing = new HashSet<>(requiredFields);
missing.removeAll(receivedFields); // difference

Set<String> all = new HashSet<>(left);
all.addAll(right); // union
```

Mutating methods change their receiver. Copy first when inputs must remain unchanged. Complexity depends on sizes and implementation; for example, membership checks in a hash set are normally cheaper than in a list.

Deduplication at System Boundaries

An in-memory Set prevents duplicates only within its own process and retention window. It does not guarantee distributed idempotency. Two service instances have different sets, and a restart loses memory.

For durable duplicate protection, use an authoritative unique database constraint, idempotency record, partitioned message key, or distributed coordination appropriate to the consistency requirement. The in-memory set may still serve as a fast optimization, but correctness must live at the right boundary.

Production Perspective

Frequent set incidents include mutable elements becoming unfindable, TreeSet comparators collapsing distinct records, nondeterministic HashSet iteration leaking into API output, huge sets used as permanent deduplication caches, and unmodifiable views mistaken for snapshots.

Diagnose the domain uniqueness rule, equality or comparator consistency, cardinality growth, retained element size, order requirements, and concurrency boundary. When a Set unexpectedly loses an object, ask whether it was rejected as equal by hash equality or comparison zero.

Interview-Ready Summary

HashSet provides expected constant-time membership and unspecified order because it is commonly backed by HashMap. LinkedHashSet adds predictable encounter order. TreeSet provides sorted $O(\log n)$ operations and treats comparator result zero as set equality. EnumSet is the compact preferred representation for enum values.

Elements must have stable equality or comparison state while stored. Unmodifiable views are not independent immutable snapshots. Concurrent set implementations protect membership operations but not cross-collection workflows. In-memory uniqueness is not distributed idempotency.

Revision Notes

- HashSet commonly stores elements as HashMap keys.
- Element equality and hashCode define HashSet uniqueness.
- HashSet gives no iteration-order guarantee.
- LinkedHashSet preserves encounter order with additional memory.
- TreeSet normally uses a red-black tree with $O(\log n)$ operations.
- TreeSet treats comparator result zero as duplicate identity.
- Add comparator tie-breakers when distinct objects must coexist.

- Comparators must be transitive, antisymmetric, and overflow-safe.
- Do not mutate fields used by hashing or ordering while stored.
- NavigableSet supports floor, ceiling, lower, higher, and range views.
- Range sets are generally backed views.
- EnumSet is compact and efficient for enum values.
- Set.of rejects duplicates and nulls.
- An unmodifiable wrapper may still expose backing-set changes.
- ConcurrentHashMap.newKeySet provides scalable concurrent membership.
- ConcurrentSkipListSet supplies concurrent sorted membership.
- Copy before destructive union, intersection, or difference operations.
- An in-memory Set is not durable distributed deduplication.
- Production correctness begins with an explicit uniqueness rule.